David King
Dr. Tong Zhang
Advanced Computer Systems
22 October 2026

Classwide Project: Cache & Memory Performance Profiling

Introduction

This report examines how cache capacity, access order, transfer size, read/write mix, and concurrent requests affect memory performance. A C benchmark measures latency and useful bandwidth across working sets from 4 KiB to 192 MiB. We also use hardware counters to relate performance to cache misses and compare normal pages with verified huge pages.

System Configuration

Item	Configuration
CPU	Intel Core i7-13800H; x86-64
Cache capacities	L1 data: 48 KiB; L2: 1.25 MiB; shared L3: 24 MiB
Memory	31.0 GiB reported by Linux
Operating system	Ubuntu 24.04.5 LTS; Linux 6.8.0-147-generic
Compiler	GCC 13.3.0
Compiler flags	-O3 -march=native -std=c11 -Wall -Wextra -Wpedantic; -pthread -lm
Frequency policy / SMT	powersave governor; SMT enabled
Page configuration	4 KiB normal pages; 2 MiB transparent huge pages via madvise
Counter access	CAP_PERFMON on the benchmark executable; counters available in all trials

Measurement Procedure

1. Pin single-thread tests to CPU 0. For the thread sweep, use CPUs 0, 2, 4, 6, 8, and 10, which belong to separate physical performance cores.
2. Start each trial in a new program process with newly allocated memory. Each worker thread stays on its assigned CPU and initializes its own memory.
3. Initialize the input data and check that each pointer ring visits every intended location exactly once before returning to the start. Before timing, run one full pass through the data to warm the cache.
4. Measure the benchmark.
5. Test each configuration 45 times, divided into three rounds of 15 trials. Randomize the order of the test conditions. Run sequential and random access tests back-to-back, alternating which goes first. For the cache-reuse comparison, do the same with the small and large buffers.
6. Validate outputs after timing and report medians and interquartile ranges across all 45 trials. Retain individual observations and per-round summaries.

Pre-experiment Expectations

Before collecting measurements, we expected random memory accesses to become slower once the data no longer fit in cache. We expected sequential access to remain faster because nearby data could be reused or fetched in advance. We expected larger transfers and more independent requests to improve throughput until another bottleneck limited performance. Huge pages were expected to reduce address-translation overhead, while repeatedly reading a small buffer that fit in cache could misleadingly suggest high DRAM bandwidth.

Experiments and Results

Unloaded Latency Baselines

Unloaded latency baselines provide a reference for how long a memory access takes before adding competing requests. We measured this using a single random pointer chain, where each load provides the address for the next, forcing accesses to occur one at a time. We selected four data sizes from the working-set sweep to represent L1, L2, L3, and DRAM access. These labels describe where the data is expected to fit however they do not prove which memory level served every load.

Median latency increased from 1.52 ns for the L1-sized data to 115.23 ns for the DRAM-sized data (Table 1 and Figure 1). These results help distinguish the effect of accessing slower memory levels from the added delays caused by contention in later experiments. We also measured core cycles per load directly instead of estimating them from an assumed CPU frequency.

Nominal regime	**Span**	**Median (ns/load)**	**IQR (ns/load)**	**Core cycles/load**
L1-sized	16 KiB	1.525	1.523–1.527	7.57
L2-sized	512 KiB	3.990	3.958–4.026	19.82
L3-sized	8 MiB	23.568	22.875–26.677	117.15
DRAM-sized	192 MiB	115.228	112.219–122.065	571.04

Table 1. Representative dependent-load latency and directly measured core cycles per load. Each row summarizes 45 trials.

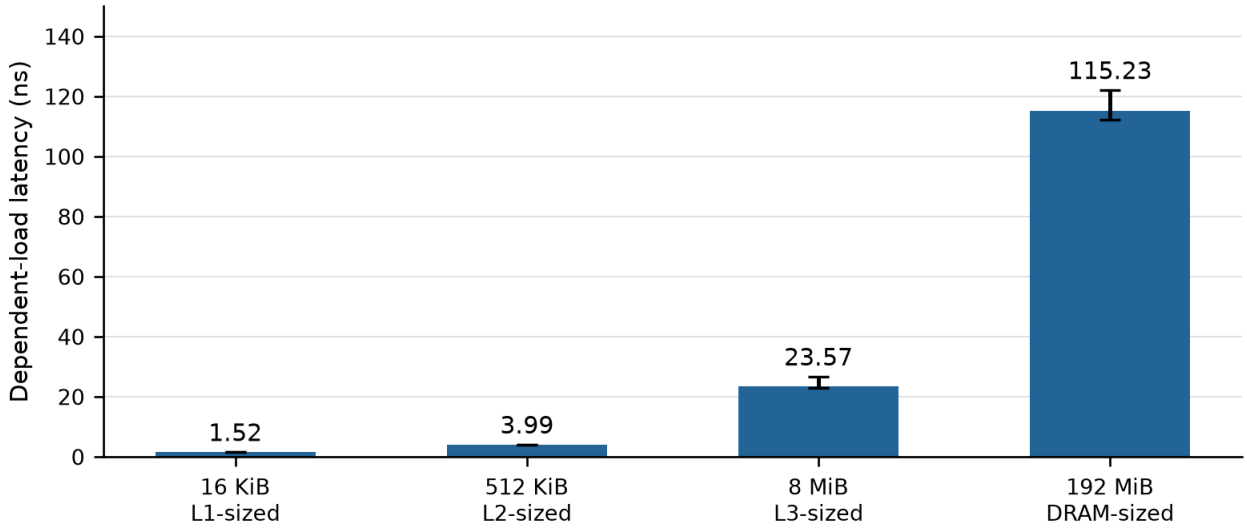


Fig. 1. Random dependent-load latency for four representative spans. Lower is faster. Bars show medians; error bars show interquartile ranges.

Access Pattern and Transfer Granularity

This experiment tested how access order, spacing and size of memory accesses affect performance. To test access patterns beyond cache capacity, we used a 96 MiB memory region, four times the size of the 24 MiB last-level cache. Within this region, we tested 13 pointer spacings from 64 to 1,024 bytes and 17 transfer sizes from 8 to 1,024 bytes. Sequential and random tests accessed the same locations in different orders. Transfer blocks were either adjacent or started four block widths apart.

Sequential pointer traversal took 4.53–13.16 ns per load, compared with 102.82–111.96 ns for random traversal (Figure 2a). Sequential access allows hardware prefetchers to predict upcoming accesses and fetch data in advance. Random access makes this prediction harder and was much slower at every spacing. However, increasing the spacing did not consistently increase random-access latency. The results reflect the combined effects of prefetching, cache-line use, and address translation.

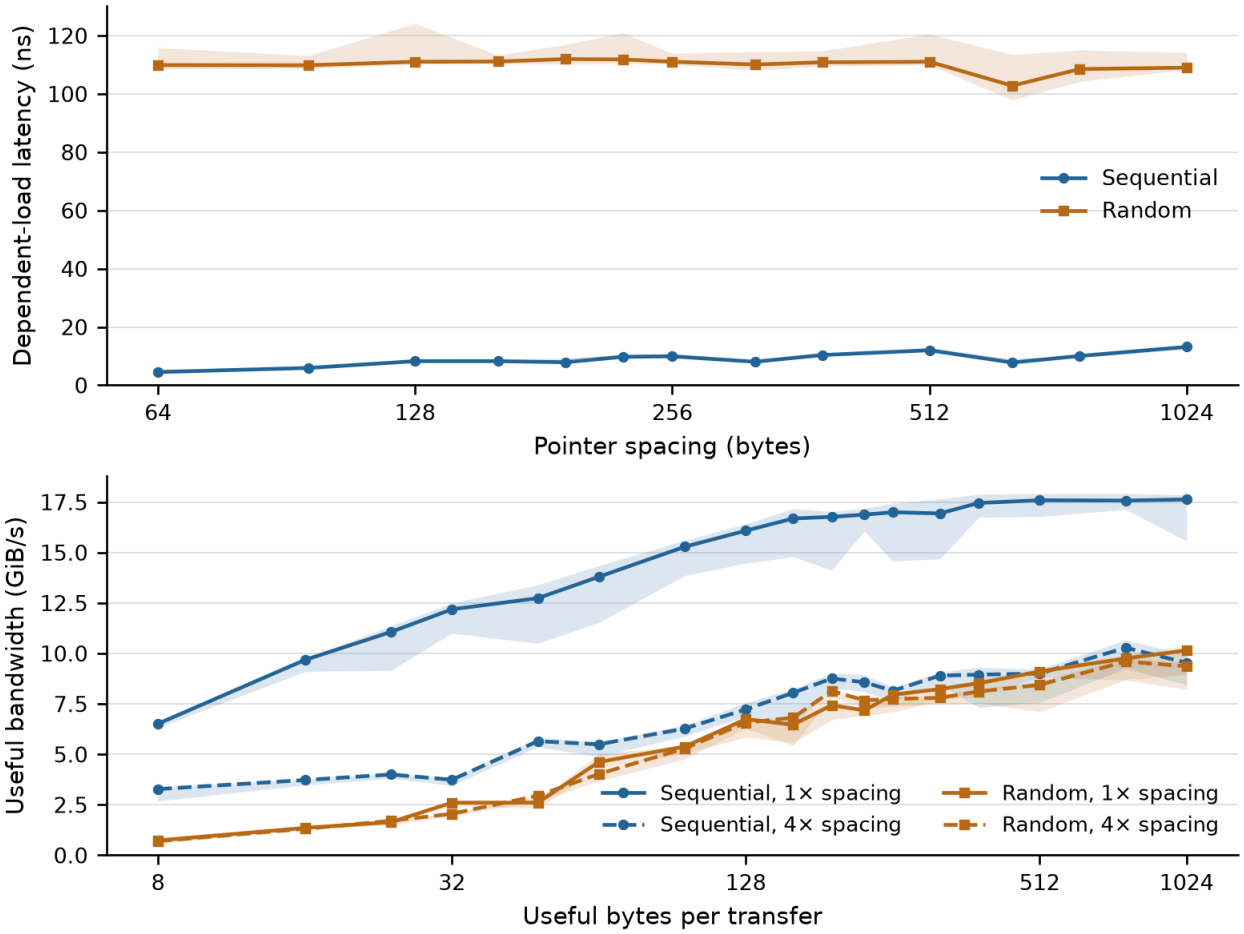


Fig. 2a. Pointer latency and useful transfer bandwidth across a 96 MiB span. Lower latency and higher bandwidth are better. Bands show interquartile ranges.

Increasing adjacent transfer size from 8 bytes to 1 KiB raised sequential bandwidth from 6.52 to 17.63 GiB/s and random bandwidth from 0.73 to 10.15 GiB/s. Larger transfers spread the overhead of each operation across more bytes and use more of each fetched cache line. Placing blocks four block widths apart generally reduced useful bandwidth, especially for sequential access. However, bandwidth did not increase consistently with each transfer-size step.

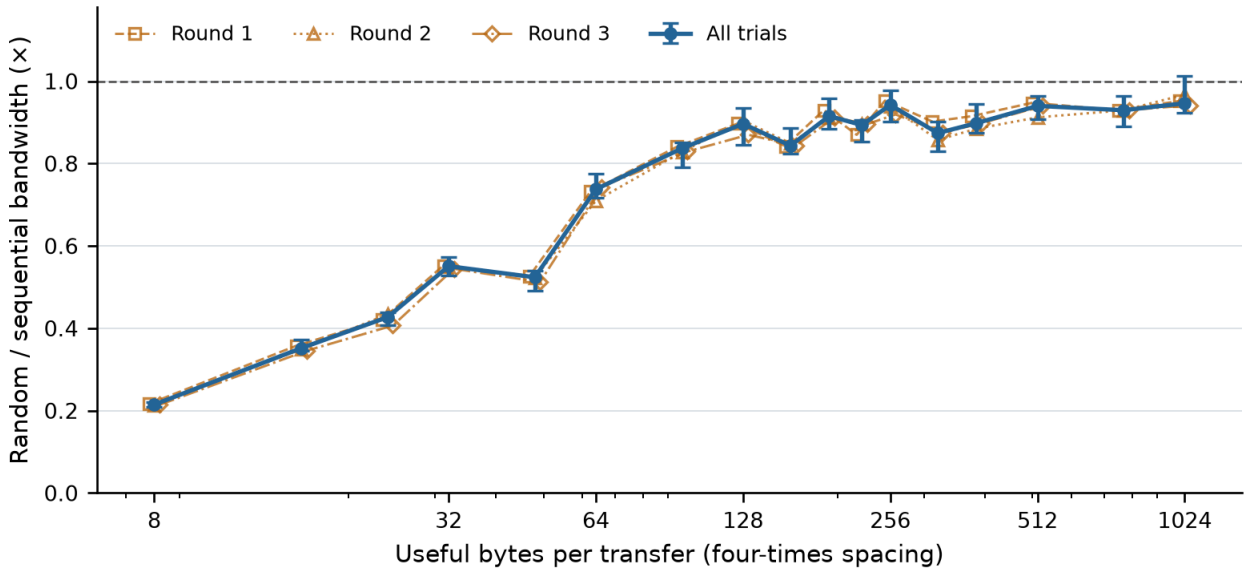


Fig. 2b. Paired random/sequential bandwidth at four-times spacing. Values below 1× favor sequential access. Error bars show interquartile ranges; open markers show round medians.

To make the random-sequential performance gap easier to compare, Figure 2b divides random bandwidth by sequential bandwidth; a ratio below 1 means sequential access is faster. The median paired ratio stayed below 1 at every transfer size. At 256 bytes, random access achieved 94.2% of sequential bandwidth, with similar results across all three rounds. At 1 KiB, it reached 94.6%, although the interquartile range crossed 1. Random access therefore showed no consistent advantage, even where the two methods performed similarly.

Read/Write Mix

We tested how the read/write balance affects bandwidth using adjacent 1 KiB blocks within a 96 MiB memory region. The 11 mixes ranged from all reads to all writes in 10-percentage-point steps. Each percentage indicates the share of blocks read or written, and we tested each mix in both sequential and random order.

For mixes of 100% reads, 70% reads/30% writes, 50% reads/50% writes, and 100% writes, sequential bandwidth was 17.45, 16.77, 14.74, and 12.28 GiB/s. Random bandwidth was 10.12, 10.36, 9.91, and 9.74 GiB/s, respectively (Figure 3). Sequential access was faster at every mix, but its advantage narrowed as the proportion of writes increased.

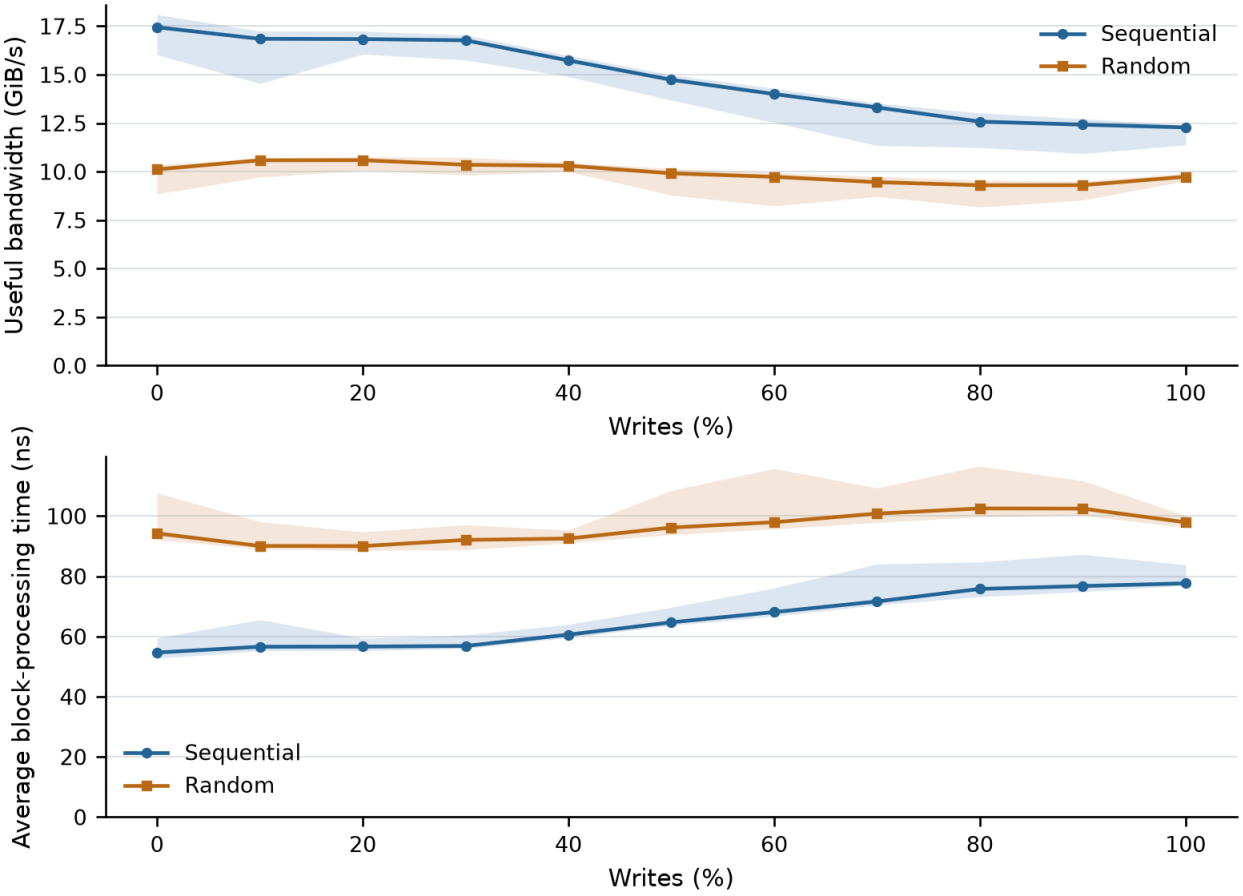


Fig. 3. Bandwidth and average block-processing time across read/write mixes. The horizontal axis gives the write percentage. Bands show interquartile ranges.

Average sequential processing time rose as the proportion of writes increased: from 54.66 ns per block for reads only to 56.86 ns for 70% reads/30% writes, 64.70 ns for an equal mix, and 77.65 ns for writes only. Processing a block therefore took about 42% longer with all writes than with all reads. These values describe the average time per completed 1 KiB block, not the latency of an individual DRAM access. Writes may require obtaining ownership of cache lines and writing modified data back to memory, helping explain the slowdown.

Concurrency and Throughput–Latency Behavior

We compared two ways to increase memory throughput: overlapping independent reads on one core and spreading reads across multiple cores. Each pointer chain requires one read to finish before the next address is known, but separate chains can make progress independently. In the single-core test, one thread took turns reading from 1–64 independent chains within one 96 MiB block of memory. In the multi-core test, one to six physical cores each ran a separate thread with

one chain and its own 96 MiB block, bringing total memory use to 576 MiB at six cores. We calculate mean chain round-trip time as active chains divided by total throughput; it represents the average interval between reads within a chain, not independently measured DRAM-access latency.

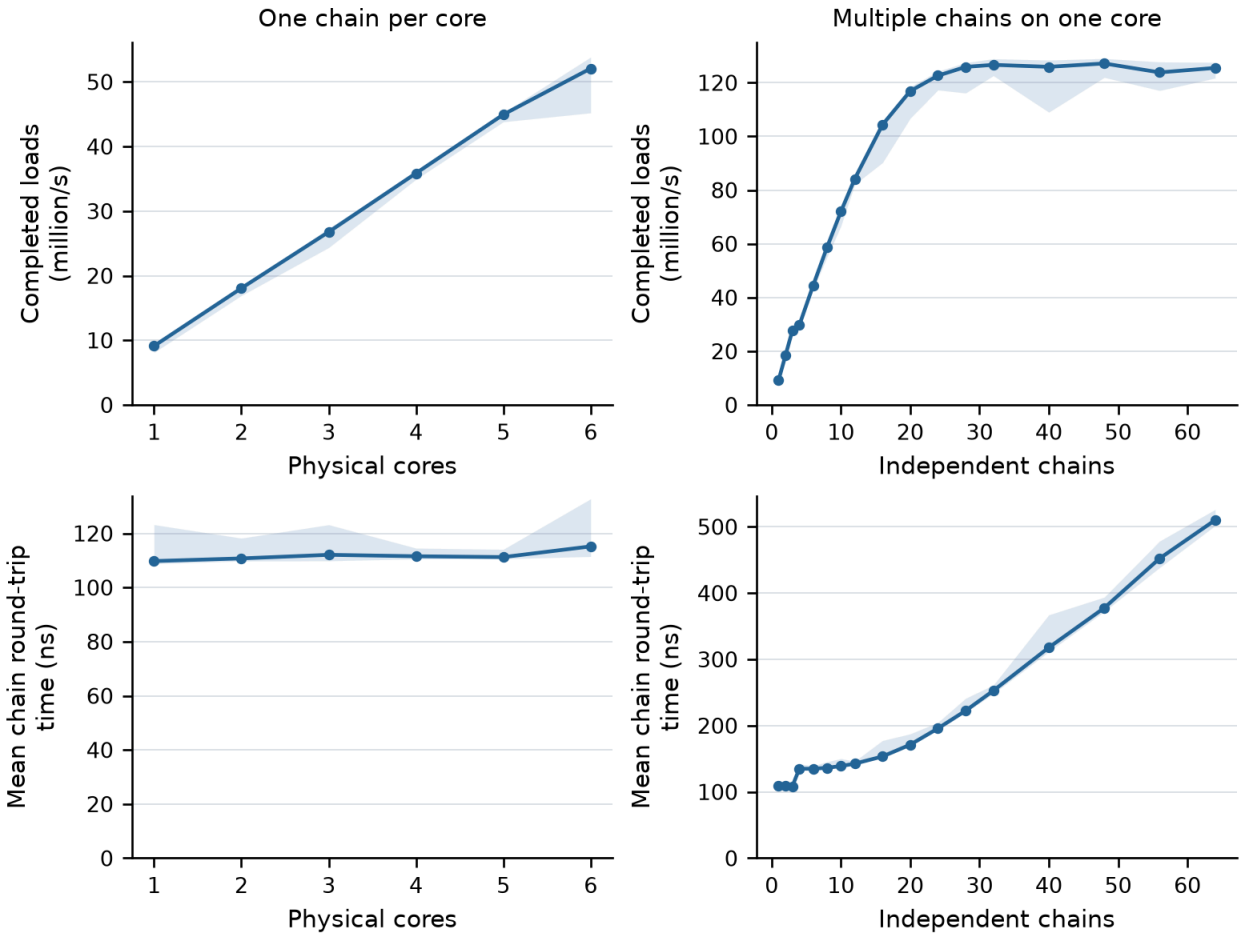


Fig. 4a. Throughput and mean chain round-trip time versus physical-core count and independent-chain count. Bands show interquartile ranges.

On one core, throughput increased from 9.12 million memory reads per second with one chain to 122.62 million with 24 chains, then leveled off at about 123–127 million through 64 chains (Figure 4a). Meanwhile, the average time between successive reads within each chain increased from 109.66 ns with one chain to 510.01 ns with 64. Multiple independent chains let the processor work on other reads while waiting for one to finish. Beyond about 24 chains, however, additional chains increased the wait before each chain could advance without substantially increasing the total number of reads completed per second.

Across one to six cores, throughput increased approximately linearly from 9.11 to 52.08 million loads/s without a clear plateau. Mean chain round-trip time remained around 110–115 ns. This differs from the one-core sweep because both the available execution resources and total footprint increased. This suggests that adding cores allowed more reads to proceed in parallel without substantially increasing the time between reads in each chain.

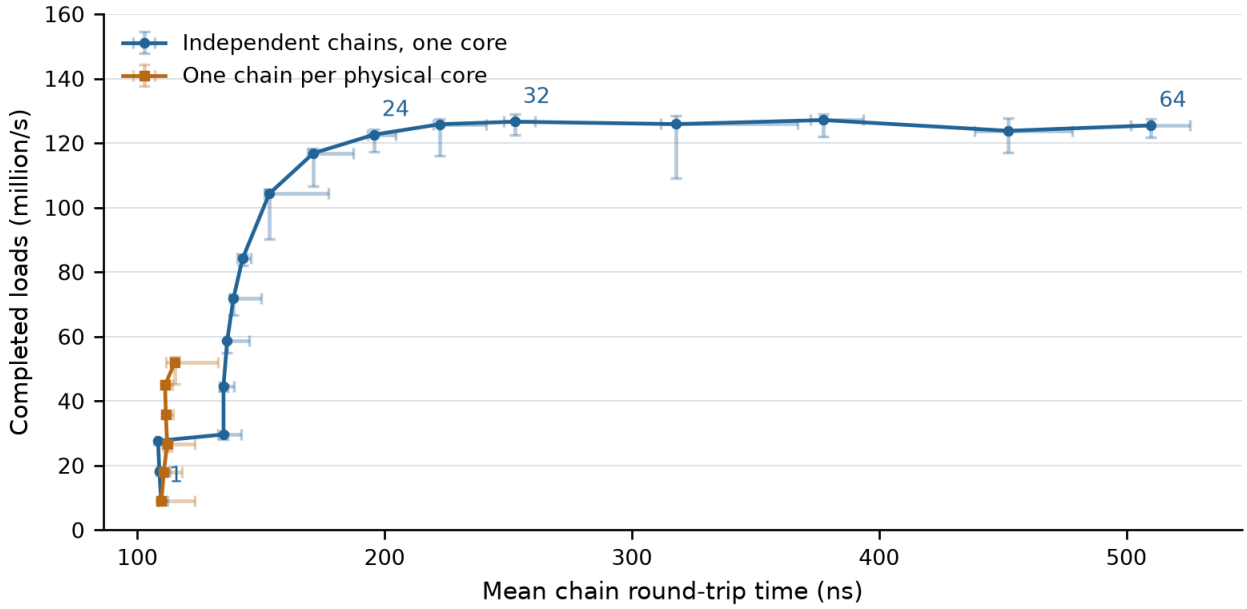


Fig. 4b. Throughput versus mean chain round-trip time. Bars show interquartile ranges on both axes; lines follow increasing concurrency. Labels identify selected chain counts.

Figure 4a shows how throughput and the time between reads change as you add more chains or cores. Figure 4b plots throughput directly against the time between reads, making the tradeoff clearer. Beyond about 24–32 chains on one core, the time between reads increases with little additional throughput. Little's Law expresses this relationship mathematically:

$$L = \lambda W$$

Here, L is the number of active chains, λ is total throughput, and W is the average time between reads within each chain. Once throughput plateaus, Little's Law implies that increasing L must increase W, explaining the diminishing returns beyond the 24–32-chain range where throughput levels off.

Locality Across Cache Levels

In this test, we changed the size of the memory buffer a pointer chain traverses to measure how access time changes as the data outgrows each cache. We tested 32 buffer sizes from 4 KiB to 192 MiB in sequential and random order, with extra measurements near the capacities of L1 (48 KiB), L2 (1.25 MiB), and the shared last-level cache (24 MiB).

Average time per random read increased from 1.79 ns at 48 KiB to 3.51 ns at 56 KiB, and from 6.17 ns at 1 MiB to 9.97 ns at 1.5 MiB. A larger, more gradual increase occurred between 8 and 32 MiB, where access time rose from 23.57 to 98.91 ns (Figure 5). At 192 MiB, random reads took 115.23 ns each, compared with 4.54 ns for sequential reads. These increases show that memory access becomes slower as the data outgrows each cache.

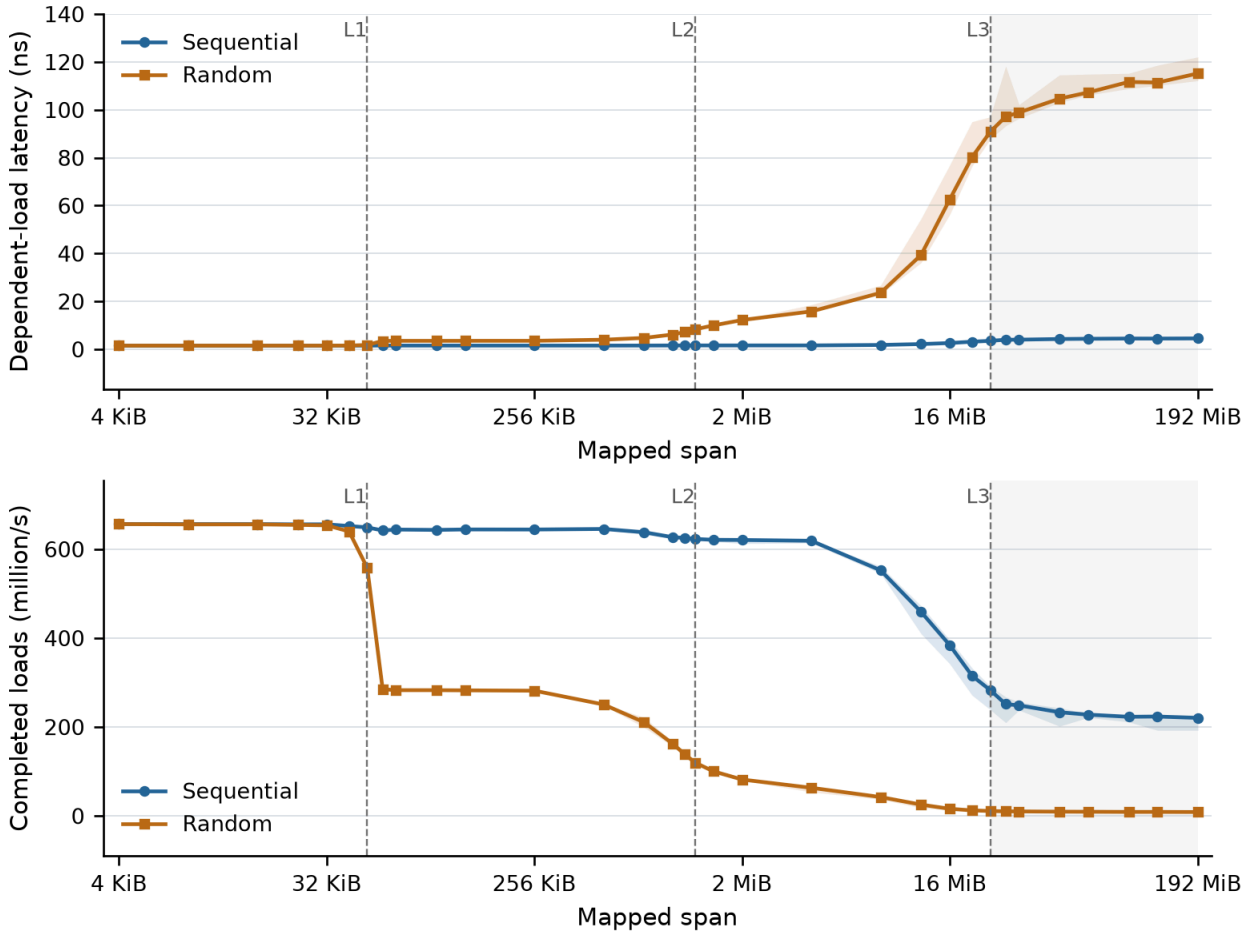


Fig. 5. Latency and completed-load throughput across working-set sizes. Bands show interquartile ranges. Dashed lines mark nominal cache capacities; shading marks spans larger than L3.

Cache-Miss Impact

We measured the components of average memory access time (AMAT) by following a random pointer chain on CPU 0, where each read determines the address of the next. Memory locations were spaced 64 bytes apart on normal pages. We tested all 32 working-set sizes 30 times, pairing runs that sampled individual load latencies with runs that measured overall access time. Hardware counters estimated the fraction of reads served by DRAM, m. The samples estimated the average time for cache-served reads, h, accounting for hits in L1, L2, and L3. The extra cost of a cache miss, p, was the average DRAM-served latency minus h.

The fraction of reads served by DRAM increased from 12.3% at 8 MiB to 83.7% at 24 MiB and 99.5% at 192 MiB. Average cache-hit time rose from about 1 ns to 3 ns and then 11–12 ns as larger working sets required access to slower cache levels (Figure 6a). Miss penalties were less reliable for small working sets because few reads reached DRAM. At large working sets, few reads were served by cache, making the apparent decline in cache-hit time uncertain.

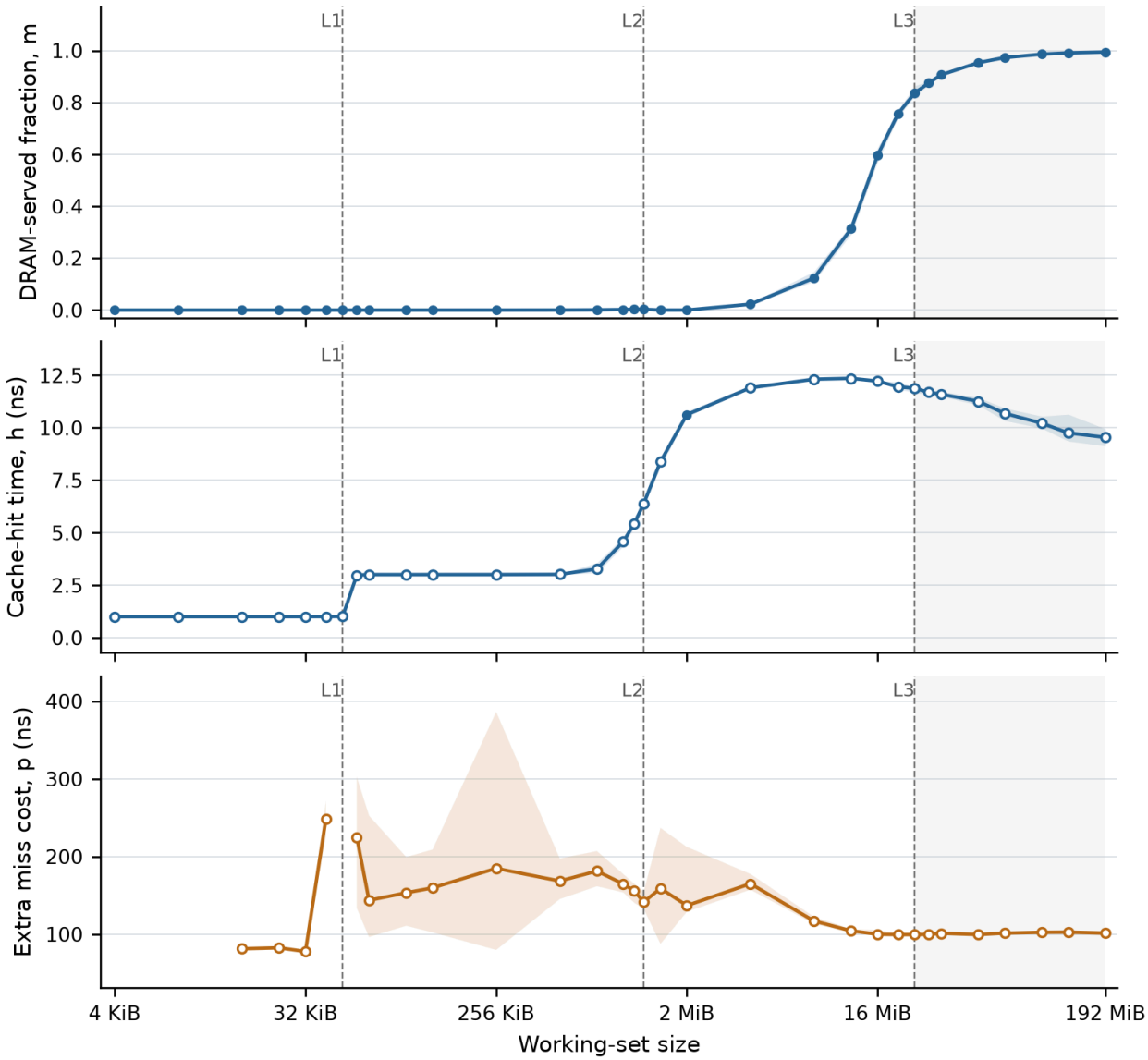


Fig. 6a. Measured full-hierarchy AMAT components. Points show medians; bands show interquartile ranges. Open markers indicate uncertain estimates; gaps indicate missing classes. Dashed lines mark nominal cache capacities.

We calculated average memory access time for each trial using AMAT = h + mp, with h, m, and p obtained from the experimental measurements. At 192 MiB, the calculated AMAT was 111.14 ns, compared with the measured 120.34 ns (Figure 6b). Both show that access becomes slower as more reads reach DRAM. Calculated AMAT and directly measured access time show similar trends. Some difference is expected because direct timing includes loop overhead, but measurement uncertainty may also contribute.

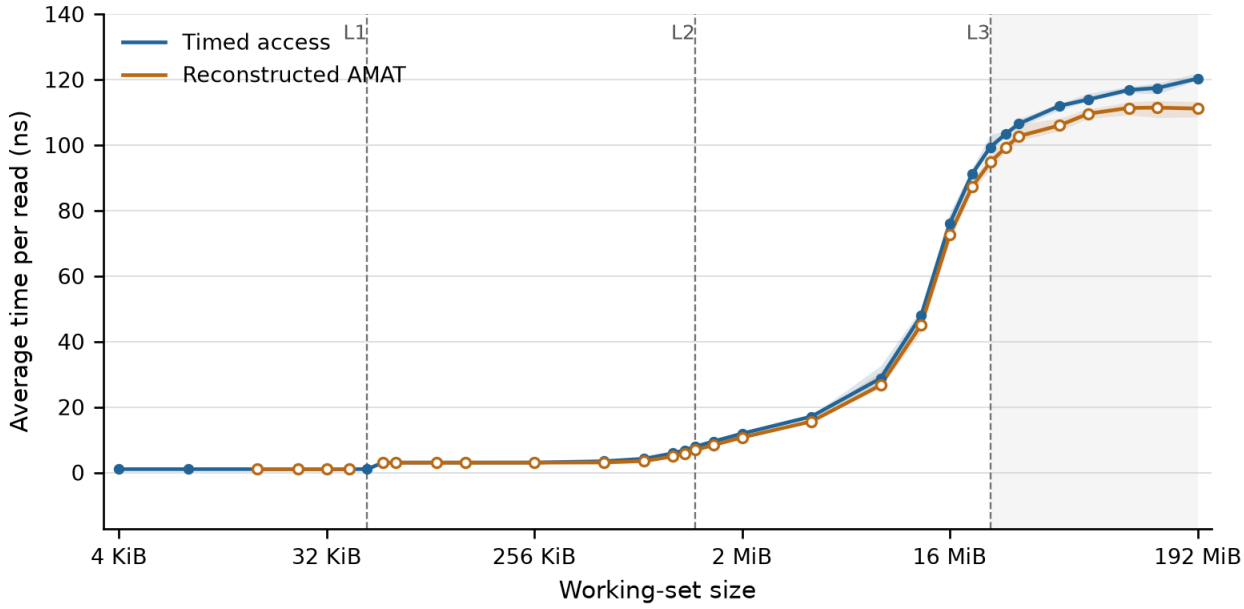


Fig. 6b. Reconstructed AMAT and independently timed access. Bands show interquartile ranges across 30 paired trials. Open markers indicate uncertain components; no values were fitted or filled in.

Overall, the results support the conclusion that memory access slows as larger working sets require more DRAM reads. The measured AMAT values follow this pattern, but some estimates are less reliable because too few cache hits or DRAM accesses were sampled.

Page Locality and Huge Pages

We tested 11 memory-region sizes from 4 to 192 MiB, reading one location every 4 KiB in either sequential or random order. For each test, we compared standard 4 KiB pages with 2 MiB huge pages and verified that huge pages were actually used. Huge pages let each cached address translation cover a larger region of memory.

At 192 MiB, huge pages lowered median access time from 113.95 to 107.35 ns for random reads and from 42.90 to 37.48 ns for sequential reads (Figure 7). However, the interquartile ranges overlap, so timing alone does not establish a consistent improvement. The hardware-counter results provide additional evidence of how huge pages affected address translation.

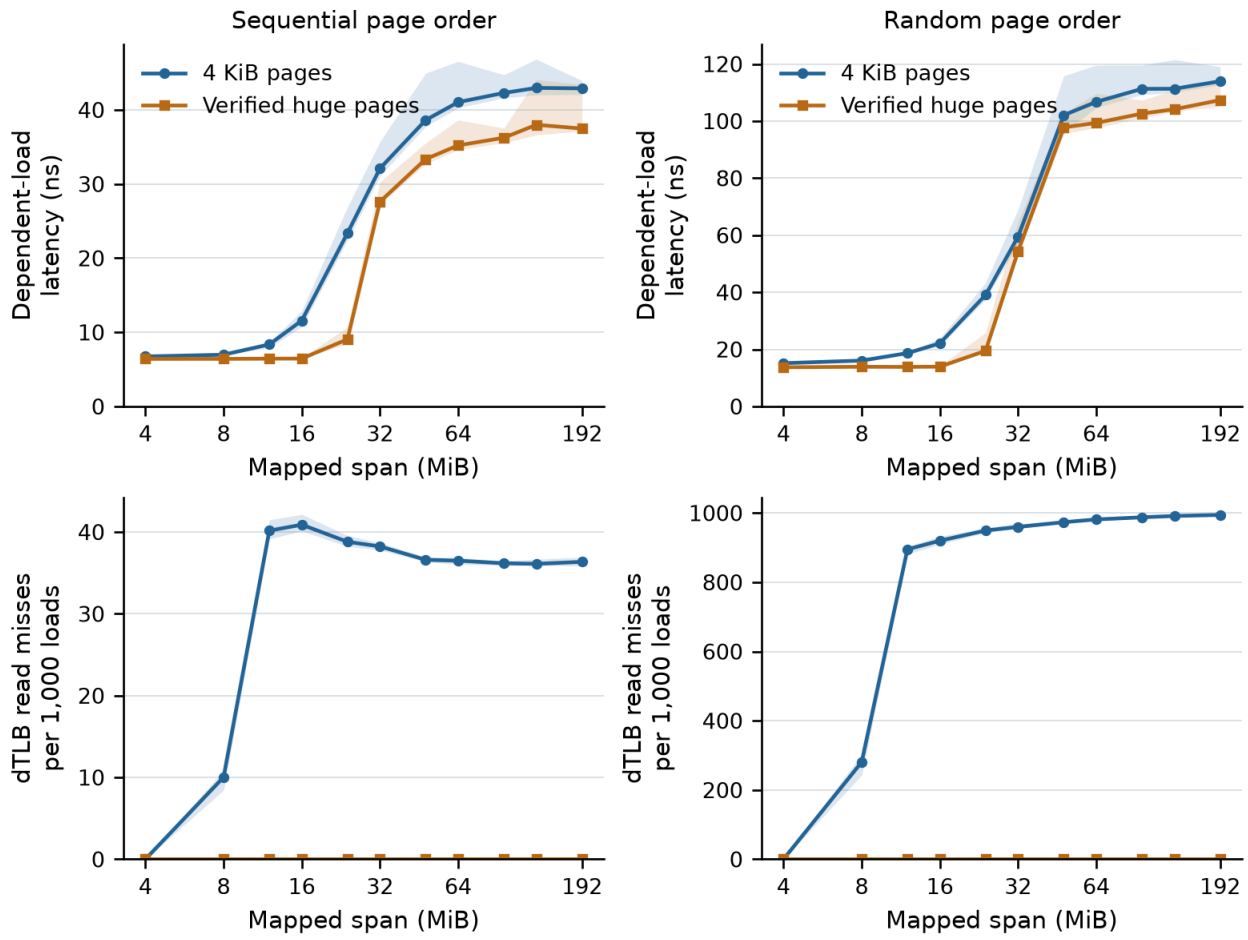


Fig. 7. Latency and dTLB read misses with normal and verified huge pages. Bands show interquartile ranges. Miss-count panels use different vertical scales.

The data translation lookaside buffer (dTLB) stores recently used virtual-to-physical memory translations. With huge pages, recorded dTLB read misses fell from 994.51 to about 0.00135 per 1,000 loads for random access, and from 36.35 to about 0.00650 for sequential access. These large reductions were accompanied by much smaller decreases in access time, suggesting that address translation accounted for only part of the overall delay.

Benchmark Pitfall: Cache Reuse

The same sequential read kernel transferred 1 KiB blocks over a reused 32 KiB buffer and a 192 MiB span. We paired small- and large-buffer trials in alternating order.

Median bandwidth was 155.31 GiB/s for 32 KiB and 17.20 GiB/s for 192 MiB (Figure 8). The median paired ratio was 8.90×; dividing the two overall medians gives 9.03×. These are different summaries of the same trials.

Repeated cache reuse produced much higher throughput than the larger working set. Reporting the small-buffer result as DRAM bandwidth would therefore be misleading. Buffer size and reuse must accompany any bandwidth claim.

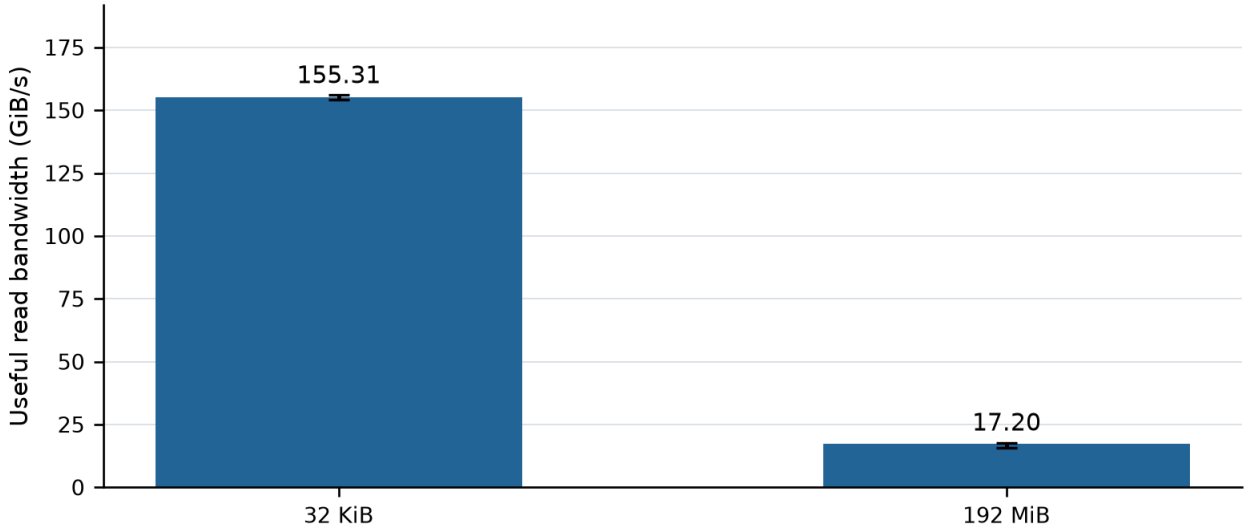


Fig. 8. Useful read bandwidth for reused 32 KiB and 192 MiB buffers. Bars show medians; error bars show interquartile ranges.

Validation and Reproducibility

All 11,205 observations across 249 configurations passed the recorded internal checks. We checked pointer rings for valid, unique traversal; bandwidth checks compared read checksums and written values against known values outside timing. Every configuration contains all 45 expected trials. We also verified counter availability, requested huge-page backing, and the saved raw-data checksum.

Source, build/run scripts, raw data, processed tables, metadata, and plotting code are retained with run 20261022T203612.685890Z. From project2-cache-memory, run make plots RUN_DIR=output/20261022T203612.685890Z to regenerate analysis from these measurements, or make all for a new acquisition. The report's grouped panels are reproduced by ../.venv-analysis/bin/python report/build_figures.py. The report refresh used Python 3.12.3, NumPy 2.5.4, and Matplotlib 3.11.2. Figure-source tables and hashes are recorded in report/figures/manifest.json.

Measurement Limits

1. Timing and variability. CPU frequency and background activity were uncontrolled. "Unloaded" describes one dependent request stream, not an idle system. Loop, timer, and translation overhead remain included. Interquartile ranges describe the middle 50% of observations, not confidence intervals.
2. Metric scope. Useful bandwidth counts requested payload, excluding write allocation and unused cache-line bytes. Block-processing time and interleaved chain round-trip time are throughput-derived; neither isolates DRAM service latency.
3. Footprints and placement. Four-times spacing uses about 24 MiB of payload within a 96 MiB span, plus the access-order array. Page tests touch one cache line per 4 KiB, so their data footprint is much smaller than their mapped span. NUMA placement was not independently measured. Nominal cache capacities also exclude space occupied by benchmark bookkeeping and other running software.
4. Counter interpretation. Events include timed-loop activity and are normalized by the number of completed benchmark loads. Counts may slightly exceed one event per load. The generic dTLB event does not represent every first-level TLB miss, and L3 counts do not provide a complete hierarchy-wide AMAT decomposition.

AI Use

Codex assisted with C code, acquisition and analysis scripts, figures, and drafting. We used pointer-ring checks, known-value comparisons, input rejection, sanitizer checks during development, and recorded-data validation to evaluate the generated work.